

Finite Element Methods in C++

G14DIS = MATH4001

Mathematics 4th Year Dissertation

2024/25

School of Mathematical Sciences

University of Nottingham

Dylan Wulfse

Supervisor: Dr. P. Houston

Assessment type: Investigation

I have read and understood the School and University guidelines on plagiarism. I confirm that this work is my own, apart from the acknowledged references.

Abstract

The finite element method (FEM) is critical to numerical analysis for solving boundary value problems (BVPs) arising in science and engineering. This project presents a comprehensive study on higher-order FEM techniques applied to second order linear and semilinear elliptic BVPs in one and two dimensions using an object-oriented approach in C++. The report establishes the fundamental mathematics, ranging from function spaces and weak formulations to error estimates, as well as the implementation of a modular FEM package. Emphasis is placed on hierarchical polynomial spaces, using integrated Legendre polynomials. The implementation includes a Picard scheme to handle a degree of nonlinearity, which is shown to achieve exponential convergence. Numerical experiments are carried out to validate theoretical error bounds under h - and p -refinement, and optimal convergence is demonstrated in one dimension, while convergence in two dimensions does not meet the optimal convergence. Applications are seen to a number of common benchmark BVPs to illustrate the range of applications.

Contents

1	Introduction	3
1.1	Aims of the Investigation	4
1.2	Main Results and Conclusions	4
1.3	Structure of the report	5
2	Literature Review	6
3	Basic Concepts	8
3.1	Function Spaces and Notation	8
3.2	Partial Differential Equations and Boundary Value Problems	9
3.3	Strong and Weak Formulations	11
3.4	Finite Difference Method	13
4	The Finite Element Method	14
4.1	Meshes and Discretisation of the Domain	14
4.2	Reference Element and Basis Functions	15
4.3	Assembly of Stiffness and Load	17
4.4	Application of Boundary Conditions	18
4.5	Existence and Uniqueness	18
4.6	System Solvers	19
5	Implementation	21
5.1	Overview	21
5.2	Element Classes	22
5.3	Utility Classes	28
5.4	Core Classes	30
5.5	Supplementary Items	40
6	Convergence Studies	41
6.1	h -refinement	41

6.2	p -refinement	42
7	Applications	43
7.1	Poisson Equation in One Dimension	43
7.2	Poisson Equation in Two Dimensions	44
7.3	Semilinear Problem	44
8	Conclusions	46
A	Code	47

1 Introduction

Throughout science and engineering, mathematical modelling mainly takes the form of differential equations, which often do not admit an analytic solution. The finite element method (FEM) addresses this by discretising the continuous domain into more manageable subdomains called *finite elements*, on which solutions are approximated locally using polynomials. Assembling all the local approximations allows FEM to construct a smooth piecewise approximation to the true solution.

This dissertation details the theoretical background, implementation, and applications of the finite element method. Specifically with a focus on solving second order linear and semilinear elliptic partial differential equations in one and two dimensions with an object oriented approach using C++.

First, a review of key finite element literature is presented to contextualise the project. The texts here establish a mathematical foundation underlying in FEM, and motivate the use of a hierarchical basis, as well as framing the need for an object oriented approach, which can flexibly address linear and semilinear problems. Following this, the preliminary materials provide theoretical background to the finite element method, such as defining function spaces and deriving weak formulations that ensure the existence, uniqueness and convergence of a FEM solution.

The bulk of the report concerns the theory and implementation of FEM: the discretisation of the domain into finite elements, the construction of high-order polynomial spaces on reference elements, and the assembly of a global system from local contributions. The object oriented implementation is broken down into logical classes, each of which encapsulates a specific function of the the FEM process. This approach was chosen for its clarity, modularity, and ease of maintenance, allowing for a flexible framework which reflects the underlying mathematical theory.

The final sections present performance analysis. Convergence studies are performed under h -refinement and p -refinement, and results are compared to theoretical error bounds. Additionally, applications are considered, including the Poisson equation in one and two dimensions, and a semilinear problem. Graphical comparisons are provided to demonstrate that the imple-

mentation accurately approximates the true solution.

Overall, this dissertation contributes a FEM framework with a focus on bridging the advanced underlying mathematics with the computational methods used to implement it, which paves the way for further research into nonlinear finite element analysis.

1.1 Aims of the Investigation

The primary objectives of the project relate to the theory and implementation of high-order finite element methods for solving second order elliptic boundary value problems. Specifically, the project aims to:

- Develop a general theoretical framework for high-order FEM, ensuring understanding of underlying function spaces, weak formulations, and error estimates guaranteeing convergence.
- Design and document a modular, object-oriented finite element package in C++ encapsulating essential components of the method from mesh discretisation and local polynomial basis evaluation to system assembly and an iterative solver.
- Develop code which is concise, efficient, and maintainable, facilitating future extensions and computational experimentation.
- Apply developed software to a variety of benchmark boundary value problems to demonstrate the solvers accurate approximation of solutions.
- Verify numerical results by comparison to theoretical error bounds through convergence studies under h - and p -refinement

1.2 Main Results and Conclusions

The main result of the project is a library of object oriented high-order finite element tools tailored to second order elliptic boundary value problems in one and two dimensions. The library is modular, and logically decomposed into self-contained classes handling distinct aspects of

the FEM workflow, for example a finite element mesh class to oversee the global assembly, and an Element class to handle local contributions.

In one dimension, the convergence studies demonstrate the error adhering to theoretical bounds, following the $\mathcal{O}(h^{p+1})$ behaviour in the L^2 -norm. Whereas the two dimensional solver exhibits appropriate convergence towards the true solution, where graphically it is demonstrated to capture the complexities of the underlying differential equation, however the convergence behaviour deviates slightly from the expected behaviour on the log-log scale. Additionally, the library incorporates a Picard scheme for iterative linearised solution of semilinear problems, which achieves the expected exponential convergence to a solution with a variety of relaxation parameters.

To summarise, the project successfully bridges the theoretical finite element foundations with a practical computational implementation. The developed library provides a scalable tool for solving a wide range of elliptic boundary value problems, converging to the true solution under mesh refinement and increasing polynomial degree.

1.3 Structure of the report

The report is structured such that a general understanding of the theory behind the finite element method is build in Section 3. This contains the functional spaces used in FEM, and the idea of weak formulations, which allow the finite element method to be built into a linear system. Following this the finite element method is described in detail in Section 4. It is shown that the domain can be discretised into finite elements, and calculations can be done locally on each element. Important results about existence of a unique solution and convergence are seen here, as well as methods for solving the linear system it forms. Continuing, Section 5 explains the object oriented design of the code. It shows the break down of the finite element method into classes which each perform a set function, which is tied back to the FEM theory. Next, Section 6 assesses the accuracy of the implementation against theoretical error bounds with respect to varying element size and varying polynomial degree. Finally, some common boundary value problems are solved in Section 7, including one and two dimensional problems and a semilinear problem.

2 Literature Review

The finite element method has long been a vital numerical technique and is an incredibly well researched area of mathematics. Fundamental contributions to FEM theory were provided by Ciarlet in *The Finite Element Method for Elliptic Problems* [1], where he rigorously formalises FEM in a mathematical context, presenting key insights into convergence properties and error estimation. These foundational ideas were extended further by Brenner and Scott in *The Mathematical Theory of Finite Element Methods* [2], emphasizing functional analysis and higher order adaptive methods, which are essential for effective implementations.

Early computational considerations and algorithmic approaches are detailed by Axelsson in *Finite Element Solution of Boundary Value Problems* [3]. He addresses common assembly and solver procedures, including iterative solvers.

Concerning higher order polynomial approximations, Szabo and Babuska advocate for hierarchical polynomial spaces in *Finite Element Analysis: Method, Verification, and Validation* [4]. This demonstrates how refinement of mesh size and polynomial degree can control modelling errors, however this is predominantly focussed on an engineering approach. Whereas, Canuto et al.[5] and Karniadakis and Sherwin[6] offer a more mathematical treatment of hierarchical bases. They introduce specific hierarchical basis functions, such as integrated Legendre polynomials, which match easily into a global basis.

As to practical implementation of elements, Solin, Segeth, and Dolezel document hierarchical triangular elements in *Higher-Order Finite Element Methods* [7]. This offers detailed guidance on the treatment of polynomial spaces on two- and three-dimensional hierarchical elements. Further algorithmic insights and implementation strategies are offered by Larson and Bengzon in *The Finite Element Method: Theory, Implementation, and Applications*[8], which bridges theoretical foundations and practical computational aspects for a range of problems.

Contemporary FEM packages, such as FEniCS Project and deal.ii, provide powerful and robust frameworks, however they can present significant complexity, which can obscure direct mathematical analyses for direct analysis of higher order techniques and convergence properties.

In summary, the mathematical and computational foundations of the finite element method

are well established in literature. However, a gap remains in directly exploring iterative nonlinear methods and high-order convergence analysis. So this investigation is motivated by developing a specialised FEM library to investigate these aspects.

3 Basic Concepts

Before delving into the finite element method, it is important to establish the fundamental concepts and notation underpinning its formulation. This section will introduce function spaces, partial differential equations and boundary value problems, and the distinction between strong and weak formulations. In addition, finite difference methods are mentioned briefly for comparative purposes.

3.1 Function Spaces and Notation

To begin with, we must define some function spaces which will be crucial throughout the report. These will be generalisations of the finite dimensional polynomial space $\mathcal{P}_p$ of all polynomials of degree less than or equal to p . Firstly, we will define a *Hilbert space* [9, pp. 131-132], which is a complete inner product space. An inner product space is a vector space H over the real numbers with an inner product $\langle \cdot, \cdot \rangle : H \times H \rightarrow \mathbb{R}$ with the following properties:

- **Symmetry:** $\langle u, v \rangle = \langle v, u \rangle$
- **Linearity:** $\langle au + bv, w \rangle = a\langle u, v \rangle + b\langle u, w \rangle$
- **Positive definiteness:** $\langle v, v \rangle \geq 0, \quad \langle v, v \rangle = 0 \iff v = 0$

Then to be a complete inner product space, a Hilbert space requires that every Cauchy sequence $\{u_n\}$ in H has $\lim_{n \rightarrow \infty} \{u_n\}$ also in H . Many of the function spaces used in the finite element method are Hilbert spaces, for example L^2 , and H^1 . The space of square-integrable functions, L^2 , is defined as

$$L^2(\Omega) := \{u : \int_{\Omega} |u(x)|^2 dx < \infty\},$$

with inner product

$$\langle u, v \rangle_{L^2(\Omega)} = \int_{\Omega} u(x)v(x) \, dx.$$

And a particularly common Sobolev space in the finite element method, H^1 , is a generalisation of L^2 such that the first weak derivatives are also square integrable, that is

$$H^1(\Omega) = \{u \in L^2(\Omega) : u' \in L^2(\Omega)\},$$

with inner product

$$\langle u, v \rangle_{H^1(\Omega)} = \int_{\Omega} u(x)v(x) \, dx + \int_{\Omega} \nabla u(x) \cdot \nabla v(x) dx.$$

We can also define $H_0^1(\Omega)$ as the set of functions in $H^1(\Omega)$ which vanish at the boundary of Ω . Finally, we define the dual space of a normed function space V , denoted V' . This is the set of all bounded linear functions on V :

$$V' = \{f : V \rightarrow \mathbb{R} : f \text{ is linear and bounded}\},$$

where f is bounded if there exists a constant $C \geq 0$ such that

$$|f(v)| \leq C \|v\| \quad v \in V.$$

Then the norm on V' is given by

$$\|f\|_{V'} = \sup_{\|v\|=1} f(v).$$

This can be used to give an idea of the size of a linear bounded functional on V , that is the maximum effect that input data can have on any element in V .

3.2 Partial Differential Equations and Boundary Value Problems

Partial Differential Equations (PDEs) are equations involving unknown multivariate functions and their partial derivatives. For an unknown function $u : \Omega \mapsto \mathbb{R}$, the k^{th} -order partial

derivative is defined as

$$F(\mathbf{x}, u, u_{x_1}, \dots, u_{x_n}, u_{x_1 x_1}, u_{x_1 x_2}, \dots, u_{x_n x_n}, u_{x_1 \dots x_1}, \dots, \underbrace{u_{x_n \dots x_n}}_{\text{order } k}),$$

where $\mathbf{x} = (x_1, x_2, \dots, x_n)$.

PDEs are broken up into three distinct types:

- **Elliptic:** These typically arise in steady-state problems, where the solution is not influenced by time. Perhaps, the simplest example is the Laplace equation:

$$\Delta u = 0.$$

The Poisson equation generalizes this by adding a source term:

$$-\Delta u = f(x, y).$$

Non-linear terms can be added too to give rise to semilinear elliptic equations, such as:

$$-\Delta u + \lambda u^{2q+1} = f(x, y).$$

- **Parabolic:** These model time-dependent processes like diffusion or heat flow. An example is the Heat equation:

$$u_t - \kappa \Delta u = f(x, t),$$

with $x \in \Omega \subseteq \mathbb{R}^n$, and $t > 0$.

- **Hyperbolic:** These model wave-like problems, for example the Wave equation:

$$u_{tt} - c^2 \Delta u = f(x, t),$$

with $x \in \Omega \subseteq \mathbb{R}^n$, and $t > 0$.

3.2.1 Initial and Boundary Conditions

A *boundary value problem* (BVP) is a PDE with boundary conditions imposed. This ensures that the solution is unique and continuously dependent on the domain, and means that the PDE problem is *well-posed*. The boundary conditions can take the form of either *Dirichlet*, or *Neumann* boundary conditions. First we will define the set of points on the boundary of the domain $\Omega \subseteq \mathbb{R}^n$ as $\partial\Omega$. Then a **Dirichlet** boundary condition requires that the solution has $u(x) = g(x)$ for all x on $\partial\Omega$; **Neumann** boundary conditions require $\frac{\partial u}{\partial \mathbf{n}}(x) = g(x)$ for all x on $\partial\Omega$. Additionally for time-dependent problems, we require an initial condition, such as $u(x, 0) = u_0(x)$ for x in Ω .

3.2.2 Importance of PDEs

PDEs are of critical importance throughout mathematics, science, and engineering. They arise naturally in many fields from Navier-Stokes equations modelling fluid flow, to Maxwell's equations describing electromagnetic fields. Analytical solutions to PDEs are often impossible to find, which motivates us to find numerical solutions, in particular the finite element method.

3.3 Strong and Weak Formulations

Before moving on to the finite element method, we will distinguish between the strong and weak formulations of a BVP. The *strong formulation* is the original form in which the full problem is stated, for example: find u in $C^2(\Omega)$ such that

$$\begin{aligned} -\Delta u &= f(x) \\ u(x) &= 0 \quad \forall x \in \partial\Omega. \end{aligned} \tag{3.1}$$

Then to find the *weak form* [8, p. 85], we let V be a Hilbert space, and multiply both sides of the strong form by a test function $v \in V$, and integrate by parts using Green's first identity:

$$\begin{aligned} - \int_{\Omega} \Delta u v &= \int_{\Omega} f v, \\ \int_{\Omega} \nabla u \cdot \nabla v - \int_{\partial\Omega} \frac{\partial u}{\partial n} v &= f v. \end{aligned} \tag{3.2}$$

Then V is chosen based on the boundary conditions required and we see that the $\int_{\partial\Omega} \frac{\partial u}{\partial n} v$ term disappears. In particular, the space required for the boundary conditions in Equation 3.1 is $V = H_0^1(\Omega)$, the Sobolev space of functions with weak derivatives in L^2 and zero boundary conditions. This completes the weak form as:

Find u in $V := H_0^1(\Omega)$ such that

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx \quad \forall v \in V$$

In general the weak form can be generalised and split up into a bilinear form $a(\cdot, \cdot)$, and a functional form $b(\cdot)$, in the case of this problem:

$$\begin{aligned} a(u, v) &= \int_{\Omega} \nabla u \cdot \nabla v \, dx, \\ \text{and } b(v) &= \int_{\Omega} f v \, dx. \end{aligned} \tag{3.3}$$

There are three main benefits to using the weak formulation when applying the finite element method to a PDE problem:

- It enforces **reduced smoothness requirements**, as H_0^1 only requires first-order differentiability compared to the second-order required by C^2 .
- It allows for **flexible handling of boundary conditions**, as the weak form naturally incorporates Dirichlet boundary conditions in the solution space, and Neumann boundary conditions arise naturally in the integration by parts step
- They are **inherently compatible with numerical methods** as it naturally gives rise to the linear system used in the finite element method.

3.4 Finite Difference Method

Finally, we consider the *Finite Difference Method*, which is discussed at length in [10]. It is a simple numerical method for solving BVPs, based on approximating derivatives using finite difference quotients derived from Taylor expansions, that is the second derivative of $u(x)$ at discrete point x_i is defined as

$$u''(x_i) \approx \frac{u(x_{i-1}) - 2u(x_i) + u(x_{i+1}))}{h^2},$$

with h as the space between points on a uniform grid. When these finite difference quotients are substituted into the PDE, a tridiagonal system is found as

$$-\frac{u_{i-1} - 2u_i + u_{i+1}}{h^2} = f_i.$$

This is a rather restrictive method for numerical solution of BVPs, as it cannot use complex, irregular domains, and there are no standard methods to increase the accuracy of the solution.

4 The Finite Element Method

The *Finite Element Method* (FEM) is one of the most common numerical methods for solving BVPs. Unlike finite difference methods, FEM subdivides the domain into smaller, simpler regions called finite elements on which polynomial approximations are made to the solution. This section will introduce key theoretical aspects of FEM as well as notation which will be used throughout the report.

4.1 Meshes and Discretisation of the Domain

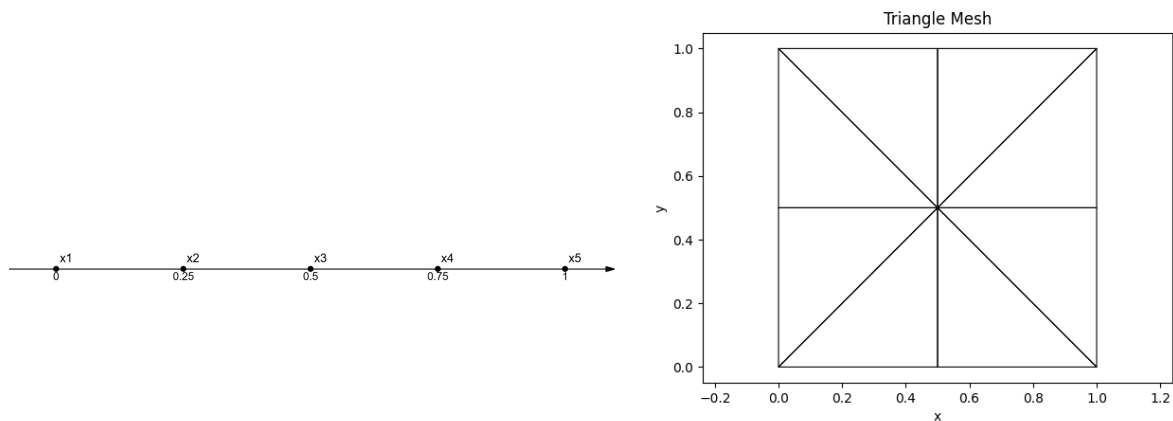
The first step to creating a FEM approximation is the discretisation of the domain. Given a PDE in the weak form

$$a(u, v) = b(v) \quad \text{for all } v \in V,$$

we define a subspace $V_h \subset V$, a finite dimensional subspace of the infinite dimensional space V . Then the *Galerkin method* [1, pp. 37-38] is: find u_h in V_h such that

$$a(u_h, v) = b(v) \quad \text{for all } v \in V_h,$$

which under certain assumptions can be shown to converge to the exact solution u , that is $u_h \rightarrow u$ in V as $h \rightarrow 0$.



(a) Regular one dimensional mesh of five elements (b) Regular two dimensional mesh of eight elements

Figure 1: Plots illustrating simple meshes in one and two dimensions

The *mesh* in a finite element method is a set of non-overlapping subdomains called elements. In one dimension, the domain $\Omega = [a, b] \subseteq \mathbb{R}$ is discretised into n interval elements $[x_i, x_{i+1}]$, such that $a = x_1 < x_2 < \dots < x_n < x_{n+1} = b$, as seen in Figure 1a. These intervals have length $h_i = x_{i+1} - x_i$, and we can set $h = \max h_i$ to represent a measure of how fine the mesh is. [11] In two dimensions, the domain $\Omega \subseteq \mathbb{R}^2$ is discretised into triangle or quadrilateral elements, seen in Figure 1b. The critical point is that in either case the set of all elements on the mesh covers the whole domain. Additionally, finite element meshes allow for a high degree of flexibility, especially in order to maximise computational efficiency. In particular, they can be irregular and they can adapt to either local geometric complexity, or the anticipated behaviour of the solution. We note however that a degree of regularity is useful to increase computational efficiency. Enforcing this can be as simple as requiring a certain ratio in the side lengths of the elements. [12]

4.2 Reference Element and Basis Functions

Most of the results in this section follow from Solin, Segeth, and Dolezel in [7]

A reference element is a representative element on which any calculations can be performed and transformed to each physical element. For example, in one dimension this could be the interval 0 to 1, or in two dimensions it could be the triangle defined by $(-1, -1)$, $(1, -1)$, $(-1, 1)$. If we call an full domain Ω defined on (x, y) , then we will call a reference element domain $\hat{\Omega}$ defined on (ξ, η) . Then for any point on the reference domain, we can define a map $F : \hat{\Omega} \mapsto \Omega$ such that $x = F(\xi)$

The *basis functions*, or shape functions, on a finite element mesh are a set of polynomials up to a given degree p on each element denoted as $\phi_i(\cdot)$ in V_h as defined earlier. Then a function $v \in V_h$ can be represented as $v(x) = \sum_{i=1}^n v(x_i)\phi_i(x)$, more particularly any v in V_h can be expressed as a linear combination of basis functions ϕ_i . This finally clarifies the role of $V_h \subseteq V := H^1(\Omega)$: where $V = \{v : v \in C^1[a, b], v(a) = v(b) = 0\}$, $V_h = \{v : v \text{ is piecewise linear}, v \in C[a, b], v(a) = v(b) = 0\}$. The basis functions are defined such that they have local support, so are only non-zero on a small set of elements. To illustrate this, we can look at the simplest basis functions in one or two dimensions, which are based on

the vertices of the element.

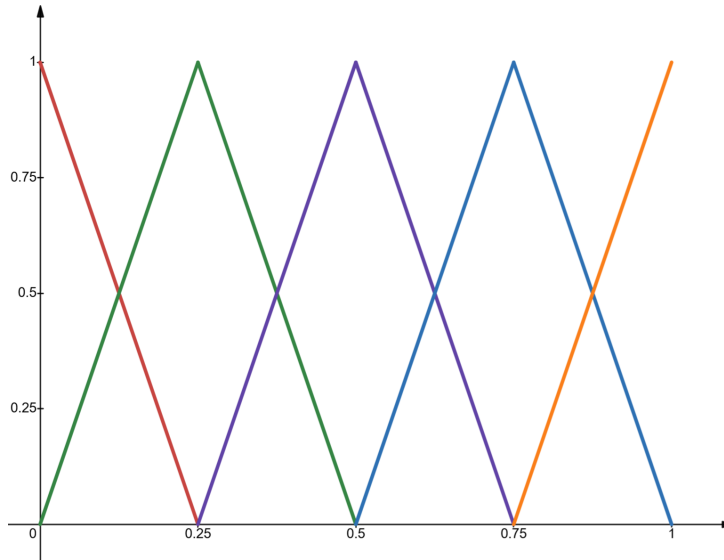


Figure 2: 1D linear basis functions

The one dimensional case can be seen in Figure 2, for example element two on subdomain $[0.25, 0.5]$ has basis functions ϕ_1 and ϕ_2 greater than zero, and all other basis functions are zero. It is fairly easy to define the linear basis functions, for example in one dimension, the linear shape functions are defined as

$$\phi_i = \frac{x_{i+1} - x}{h}, \quad \phi_{i+1} = \frac{x - x_i}{h}, \quad h = x_{i+1} - x_i.$$

There can be any number of basis functions, and the basis used can be either nodal or modal. In the case of a nodal basis, there are extra nodes added on each edge of the element and polynomials degree p are defined across the nodes such that they are 1 at node i , and 0 at all other nodes. On the other hand, a modal basis can be more effective for higher-order approximations, it works by adding polynomial modes for all degrees up to degree p , for example Legendre or Lobatto polynomials.

All basis functions used can be found in Section 5.2.

4.3 Assembly of Stiffness and Load

In the finite element method, the continuous weak formulation

$$a(u, v) = b(v),$$

is discretised by approximating u by u_h in the finite dimensional subset $V_h \subset V$. We represent this as we substitute in the approximations

$$u_h(x) = \sum_{j=1}^n u_j \phi_j(x),$$

with basis functions $\{\phi_i\}_{i=1}^n$ in V_h , and unknown coefficients u_j . by selecting test function $v = \phi_i(x)$ in V_h , the weak formulation becomes

$$a(u_h, \phi_i) = b(\phi_i).$$

This results in the linear system

$$Au = b \quad \text{with} \quad A_{ij} = \int_{\Omega} \nabla \phi_j(\mathbf{x}) \cdot \nabla \phi_i(\mathbf{x}) \, d\mathbf{x}, \quad b_i = \int_{\Omega} f(\mathbf{x}) \phi_i(\mathbf{x}) \, d\mathbf{x}.$$

Then using the local support of the basis functions, the integrals can be decomposed into contributions from individual finite elements. If the domain Ω is partitioned into elements $\{\kappa\}$, then

$$A_{ij} = \sum_{\kappa} \int_{\kappa} \nabla \phi_j(\mathbf{x}) \cdot \nabla \phi_i(\mathbf{x}) \, d\mathbf{x},$$
$$b_i = \sum_{\kappa} \int_{\kappa} f(\mathbf{x}) \phi_i(\mathbf{x}) \, d\mathbf{x}.$$

This element-wise assembly is crucial as it transforms the continuous problem into a sparse linear system, which can be solved efficiently using linear algebra techniques.[8, pp. 25-26].

This also yields an important result that the matrix A is symmetric and positive definite, which is proven in [8, p. 75]

4.4 Application of Boundary Conditions

Recalling the types of boundary conditions in Section 3.2.1, we can consider how to apply them in the matrix system. For a Dirichlet boundary condition, we identify all nodes along the boundary, then set the corresponding row/column entries in the stiffness matrix to zero, and the diagonal to one. Finally set the load vector to the required value, which is often zero. For a Neumann boundary condition, the stiffness matrix does not need alteration, and we alter the load vector for that entry such that

$$b_i \mapsto b_i + \int_{\delta\Omega_N} h\phi_i.$$

4.5 Existence and Uniqueness

To motivate the use of FEM to approximate the solution to BVPs, we must ensure the problem is *well-posed* [1, pp. 7-9]. For this to be the case, we require that the solution exists, is unique, and depends continuously on the given data. A crucial result addressing this is the *Lax-Milgram Lemma*, defined in [3]. Firstly, we recall results from Section 3.3, and define a Hilbert space V , with inner product $\langle \cdot, \cdot \rangle_V$ and norm $\|\cdot\|_V$. Then we consider a weak formulation of the BVP with bilinear $a(\cdot, \cdot)$, and linear functional $b(\cdot)$. Then we require the bilinear form to satisfy:

1. **Continuity**: there exists a constant β such that

$$|a(u, v)| \leq \beta \|u\|_V \|v\|_V, \quad u, v \in V$$

2. **Coercivity**: there exists a positive constant ρ such that

$$a(u, u) \geq \rho \|u\|_V^2, \quad u \in V$$

Additionally, we require the functional form is **bounded**; that is there exists a constant $\delta > 0$ such that

$$|b(u)| \leq \delta \|u\|_V.$$

Under these conditions, the Lax-Milgram Lemma guarantees the following:

Theorem 1 (Lax-Milgram Lemma) *There exists a unique $\hat{u} \in V$ such that*

$$a(\hat{u}, v) = b(v) \quad \forall v \in V.$$

This is a significant result for a number of reasons. Firstly, it guarantees that the weak formulation of the BVP is well-posed; especially, even if a BVP does not have a classical solution it can still have a weak solution in an appropriate Sobolev space. Second, since FEM approximates infinitely dimensional V by finite dimensional V_h , the Lax-Milgram lemma ensures the corresponding finite element problem also has a unique solution. Moreover, the theorem allows a stability estimate to be found to ensure the solution will not grow unbounded relative to the input data. For this we recall the definition of the norm on the dual space V' from Section 3.1:

$$\|f\|_{V'} = \sup_{\|v\|=1} |f(v)|.$$

Then we can say

$$\|\hat{u}\|_V \leq \frac{1}{\rho} \|b\|_{V'}.$$

This represents the continuous dependence of the solution on the input data, that is if the dual norm of the data is small, the solution is also small.

4.6 System Solvers

In the finite element method, the assembly process forms large sparse systems of equations in the form

$$Au = b,$$

with global stiffness matrix A being symmetric and positive-definite. Solving this type of system is a cornerstone of the finite element method. The methods are broken down into direct solvers and iterative solvers.

4.6.1 Direct Solvers

Direct solvers factorise the matrix A by a method like LU or Cholesky, then solve the system by forward and backward substitution. LU factorisation is carried out by finding the decomposition of A as $A = LU$, where L and U are upper and lower triangular matrices respectively. Then the solution is found by solving $Ly = b$ for y , then solving $Ux = y$ for y . This is a robust method, with a proof provided in [13, pp. 50-51] that the factorisation does not break down due to division by zero. Cholesky factorisation is specifically designed for symmetric, positive definite matrices. It decomposes the matrix as $A = LL^T$, and similarly to Lu solving $Ly = b$ for y , then $L^Tx = y$ for x . This is a lower cost method than LU when applicable, and similar numerical stability [14].

4.6.2 Picard iteration

In the case of semilinear BVPs, both a linear and a non-linear term are included, meaning the system cannot be solved using a direct solver. Instead, there are a number of iterative solvers which can be used such as Newton's method, or a fixed-point iteration. However, a common method is a *Picard scheme* [15], which starts from an initial guess U_0 , and iterates with a relaxation parameter $0 < \alpha < 1$ as:

$$a(U_{n+1}, v) = (1 - \alpha)a(U_n, v) + \alpha\left(\int_{\Omega} f v \, d\mathbf{x} + \lambda b(U_n; v)\right). \quad (4.1)$$

This is then solved using a direct solver at each step to generate a sequence of approximations $\{U_n\}_{n \geq 1}$. This method effectively linearises the problem at each step by treating the $b(U_n; v)$ as known using the previous iterate.

5 Implementation

This section will break down the implementation of the finite element method in C++. It will clearly define and justify all decisions made, and demonstrate natural links between the theory and the algorithms implemented. Firstly, we will see an overview description of how the program works, with a class diagram to show how the classes interact. Following this, the classes in the program are broken down into three distinct types: core classes, element classes, and utility classes. The core classes are the main classes used to control the overall execution of the finite element method, element classes relate to individual elements and their basis functions, and utility classes are a collection of general purpose routines and structures used throughout the code. Finally, there will be a brief section on third-party software used for direct methods and mesh generation.

5.1 Overview

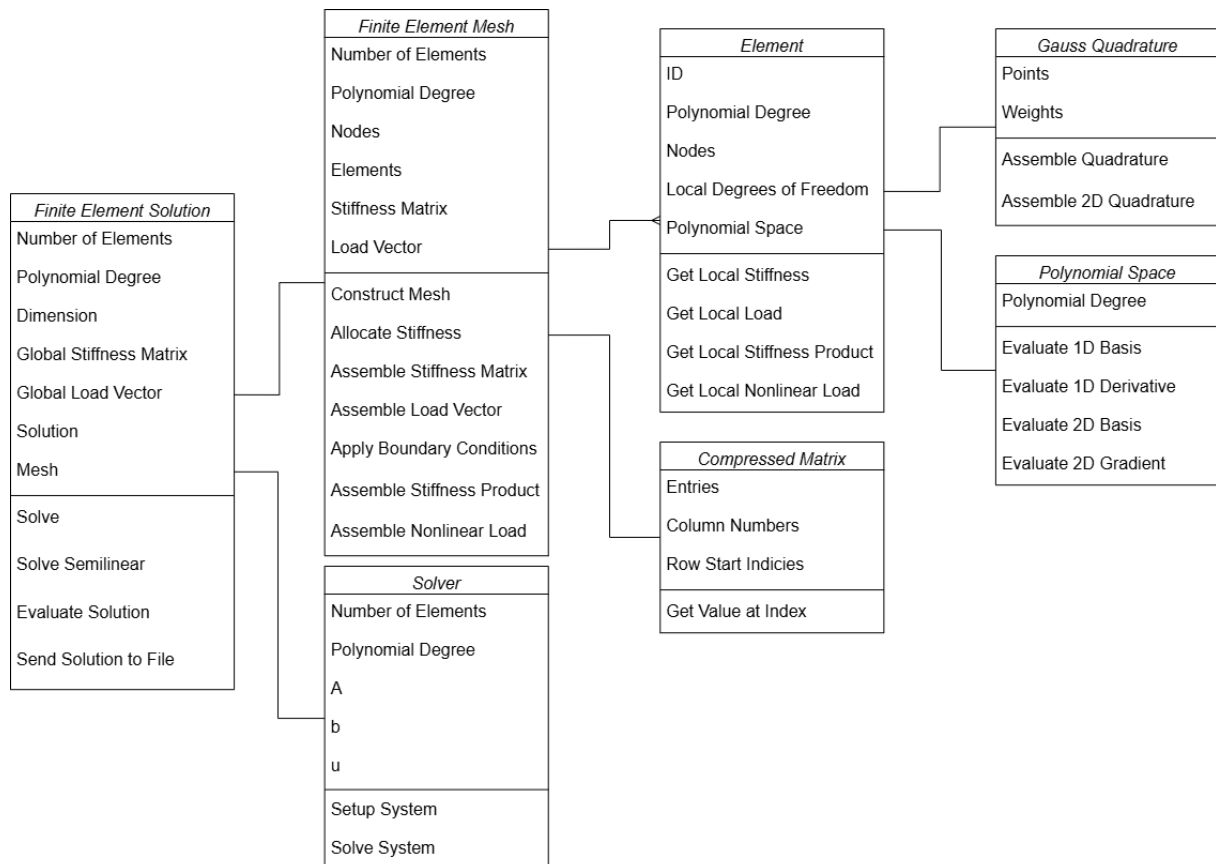


Figure 3: High level class diagram

We begin by looking at the overall structure of the implementation, show in Figure 3. The finite element solution class acts as a controller for the program, abstracting the entire solution process into a solve (or solve semilinear) function, as well as allowing for evaluation and error analysis. This class holds two more important high level classes: the finite element mesh, and the solver, handling construction of the problem and solution respectively. The finite element mesh holds the elements and the nodes they are defined on, in addition to the stiffness matrix and load vector. Then each element holds a set of nodes, its degrees of freedom, and a polynomial space, which is evaluated to calculate the local stiffness matrix and load vector. The Gaussian quadrature allows an n -point quadrature to be constructed, and the compressed matrix allows for efficient storage of sparse matrices.

5.2 Element Classes

The finite element method relies on subdividing the domain into elements, on which local polynomial spaces approximate the solution. In this section, we will focus on the element and polynomial space classes, which encapsulate the core mathematical operations of the finite element method.

5.2.1 One and Two Dimensional Elements

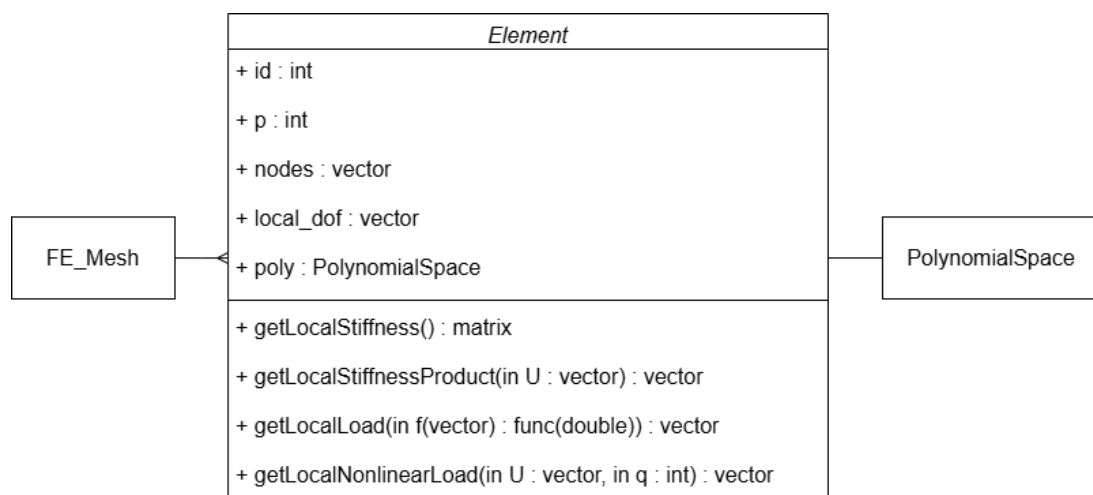


Figure 4: Element Class Diagram

The *Element* class represents the building blocks of the finite element method. It en-

capsulates an identifier, a polynomial degree (p), and a polynomial space. It also contains a set of nodes with a corresponding vector of degrees of freedom for the connectivity of the element, which is enriched with extra hierarchical degrees of freedom representing the higher order polynomial bases. It handles local computations of stiffness and load by performing integrals of the polynomial basis functions.

Computation of Local Stiffness and Load The methods `getLocalStiffness` and `getLocalLoad` approximate the bilinear and linear forms

$$a(u, v) = \int_{\kappa} \nabla u \cdot \nabla v \, d\mathbf{x}, \quad b(v) = \int_{\kappa} f v \, d\mathbf{x},$$

on each element κ . These are both found through numerical integration using Gaussian quadrature on a reference element to integrate all basis functions up to degree p .

In One Dimension The reference stiffness is calculated numerically on the reference element $[-1, 1]$ using an $n_q := p + 1$ point Gaussian quadrature as

$$\begin{aligned} A_{ij}^{\text{ref}} &= \int_{\kappa_{\text{ref}}} \phi'_i(\xi) \phi'_j(\xi) \, d\xi \\ &= \sum_{k=1}^{n_q} w_k \phi'_i(\xi_k) \phi'_j(\xi_k), \end{aligned} \tag{5.1}$$

where $\{\xi_k, w_k\}$ are the points and weights of the Gaussian quadrature. This is then transformed to the physical element $\kappa_i = [x_i, x_{i+1}]$ via an affine transformation. Computationally, this means scaling by the Jacobian of the transformation $J = \frac{x_{i+1} - x_i}{2}$:

$$A_{ij}^{\kappa} = \frac{1}{J} A_{ij}^{\text{ref}}.$$

The local load vector is calculated similarly as

$$\begin{aligned} b_i &= \int_{\kappa} f \phi_i \, dx \\ &= \sum_{k=1}^{n_q} w_k f(F(\xi_k)) \phi_i(\xi_k), \end{aligned} \tag{5.2}$$

where $F(\xi_k)$ represents the transformation of the quadrature point to the physical domain, and J is the Jacobian scaling of the integration.

This structure handles a space enriched with bubble modes on additional degrees of freedom effectively - their contributions are handled in the same way as the nodal basis functions.

In Two Dimensions These ideas are extended to triangular elements, with a reference element is defined by $\{(\xi, \eta) \in \mathbb{R}^2 : -1 < \xi, \eta; \xi + \eta < 0\}$. Again a crucial factor is the affine transformation $F : \kappa^{\text{ref}} \rightarrow \kappa$, which is defined by a matrix D , then local gradients can be evaluated as

$$\nabla \phi_{\text{phys}} = D^{-T} \nabla \phi_{\text{ref}}.$$

An n_q -point quadrature is defined on the reference element, and the local stiffness matrix is evaluated on the physical element as

$$A_{ij}^{\kappa} = \sum_{k=1}^{n_q} w_k (\nabla \phi_i^{\text{phys}} \cdot \nabla \phi_j^{\text{phys}}) |D|.$$

At this point, an adjustment is made to account for shared edges between elements. This is done by considering the global numbering of the two vertices forming the edge, then if the vertex numbers are not in increasing order, a -1 multiplier is applied to the corresponding edge basis functions. Then the load is found as

$$b_i = \sum_{k=1}^{n_q} w_k f(F(\xi_k, \eta_k)) \phi_i(\xi_k, \eta_k) \frac{|D|}{4}$$

For Semilinear Problems There are two more methods used during the iterative solution of semilinear problems, `getLocalStiffnessProduct` and `getLocalNonlinearLoad`, both

depending on the current solution U . The local stiffness product finds the bilinear form:

$$\begin{aligned}
a_\kappa(U_n, v) &= \int_\kappa \nabla u_n \cdot \nabla v \, d\mathbf{x} \\
&= \int_\kappa \left[\sum_{j=1}^{N_{\text{dof}}} U_j \nabla \phi_j \right] \cdot \phi_i \, d\mathbf{x} \\
&= \sum_{k=1}^{n_q} w_k \left[\sum_{j=1}^{N_{\text{dof}}} U_j \nabla \phi_j \right] \cdot \phi_i |D|,
\end{aligned} \tag{5.3}$$

where the final step follows from applying an n_q -point Gaussian quadrature over the reference triangle, and U_j is used as a shorthand for the value of U at the position `local_dof[j]`. Then the local non-linear portion of the load vector $b(U_n; v)$ is found as

$$\begin{aligned}
b_\kappa(U_n; v) &= \int_\kappa U_n^{2q+1} v \, d\mathbf{x} \\
&= \int_\kappa \left[\sum_{j=1}^{N_{\text{dof}}} U_j \phi_j \right]^{2q+1} \phi_i \, d\mathbf{x} \\
&= \sum_{k=1}^{n_q} w_i \left[\sum_{j=1}^{N_{\text{dof}}} U_j \phi_j \right]^{2q+1} \phi_i \frac{|D|}{4},
\end{aligned} \tag{5.4}$$

where again the $U_j \phi_j$ term corresponds to the local degrees of freedom at j , representing the evaluation of the current iterative solution at the quadrature point. Additionally the q in the power is a term passed into the function as specified for the particular problem.

5.2.2 Polynomial Space

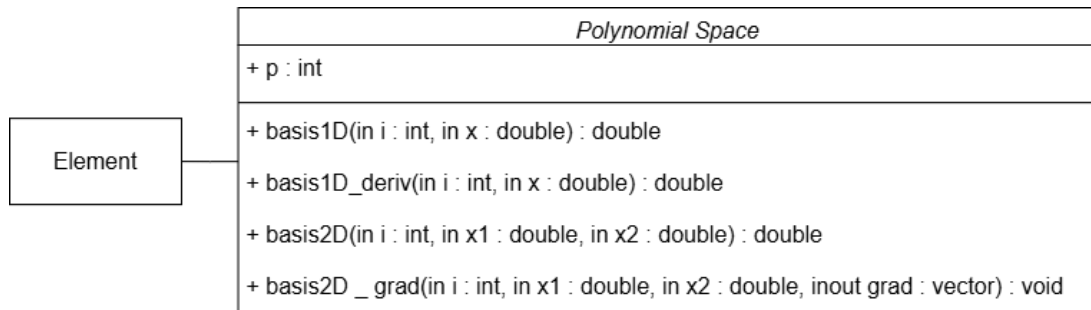


Figure 5: Polynomial space class diagram

The `PolynomialSpace` class is used to evaluate the set of polynomial basis functions on a reference element in one or two dimensions, controlled by the parameter p , which specifies the

maximum polynomial degree to use. Overall this is instrumental in defining the finite element space V_h approximating the continuous solution to the BVP.

Modal and Nodal Bases Two types of basis used in the finite element method are modal and nodal bases. In a nodal basis, extra nodes are created on the element and the basis functions are each associated with one of the nodes and defined such that $\phi_i(x_j) = \delta_{ij}$. Then interpolation is performed directly at these extra nodes. In contrast, a modal basis expresses the solution as a linear combination of modes defined hierarchically such that the space with $p - 1$ modes is completely contained within the p space.

For this work, a modal basis will be used due to its hierarchical formulation which is well-suited for higher order approximations, especially in two dimensions [7, pp. 4–7]. Additionally it leads to a less expensive stiffness matrix computation cost according to Karniadakis [6, p. 47].

In One Dimension The polynomial space is built from vertex and bubble functions defined on the reference interval $[-1, 1]$. The vertex functions are given by

$$\phi_1(\xi) = \frac{1 - x}{2}, \quad \phi_2(\xi) = \frac{x + 1}{2},$$

ensuring interpolation $\xi = \pm 1$. The space is then enriched with $p - 1$ bubble modes based on Lobatto polynomials. Specifically these are defined recursively by:

$$l_0(x) = \frac{1 - x}{2}, \tag{5.5}$$

$$l_1(x) = \frac{x + 1}{2}, \tag{5.6}$$

$$l_k(x) = \frac{1}{||L_{k-1}||_2} \int_{-1}^x L_{k-1}(\xi) d\xi, \quad 2 \leq k, \tag{5.7}$$

where $||L_{k-1}||_2 = \sqrt{2/(2k - 1)}$, and L_k represents the Legendre polynomial of degree k . This modal structure allows for hierarchical decomposition, where bubble modes do not influence nodal values, but enrich the internal approximation.

Properties of Lobatto Polynomials Lobatto polynomials have a number of desirable properties for finite element approximations, in particular they vanish at ± 1 . This leads to well-localised global basis with vertex functions non zero at exactly one endpoint, and bubble modes with local support only vanishing at the endpoints allowing for easy matching to neighbouring elements. [5]

In Two Dimensions The hierarchical basis functions follow the approach detailed by Solin [7, pp. 56-61]. The reference triangle is defined as

$$\{(\xi, \eta) \in \mathbb{R}^2 : -1 < \xi, \eta; \xi + \eta < 0\},$$

with affine coordinates

$$\lambda_1(\xi, \eta) = \frac{\eta + 1}{2}, \quad \lambda_2(\xi, \eta) = -\frac{\xi + \eta}{2}, \quad \lambda_3(\xi, \eta) = \frac{\xi + 1}{2}.$$

The vertex functions are defined as a permutation of these coordinates such that unit values are achieved at the respective vertex.

$$\begin{aligned} \phi^{v_1}(\xi, \eta) &= \lambda_2(\xi, \eta) \\ \phi^{v_2}(\xi, \eta) &= \lambda_3(\xi, \eta) \\ \phi^{v_3}(\xi, \eta) &= \lambda_1(\xi, \eta) \end{aligned} \tag{5.8}$$

For polynomial degrees $p \geq 2$, edge functions are added. For each edge, these coincide exactly with the one dimensional Lobatto polynomials along the edge and are zero at the other two edges.

$$\begin{aligned} \phi_k^{e_1} &= \lambda_2 \lambda_3 \varphi_{k-2}(\lambda_3 - \lambda_2), \\ \phi_k^{e_2} &= \lambda_3 \lambda_1 \varphi_{k-2}(\lambda_1 - \lambda_3), \\ \phi_k^{e_3} &= \lambda_1 \lambda_2 \varphi_{k-2}(\lambda_2 - \lambda_1), \end{aligned} \tag{5.9}$$

where φ_{k-2} is the kernel function, defined such that $l_k(x) = l_0(x)l_1(x)\varphi_{k-2}(x)$, $2 \leq k$. When considering edge modes in practice, it is critical to ensure that the edge functions on a shared global edge will line up, particularly for odd numbered polynomial degrees. This is typically handled by ensuring shared edges are always numbered from low to high global degrees of freedom, and applying a -1 multiplier if not. Finally if polynomial degree $p \geq 3$, bubble functions are added. They are defined to vanish completely on all element boundaries, thus there is no concern as to compatibility between elements. They once again incorporate the Lobatto shape functions through the kernel functions.

$$\phi_{n_1, n_2}^b = \lambda_1 \lambda_2 \lambda_3 \phi_{n_1-1}(\lambda_3 - \lambda_2) \phi_{n_2-1}(\lambda_2 - \lambda_1) \quad (5.10)$$

5.3 Utility Classes

Before moving on to the core classes, which conduct the finite element method at a high level, we will introduce the utility classes - a collection of general purpose routines and structures which are crucial to supporting the implementation. These include a compressed sparse row matrix for efficient storage of sparse matrices, Gaussian quadrature for numerical integration, and a selection of geometric methods handling mapping of points and calculation of affine transformation matrices.

5.3.1 Compressed Sparse Row Matrix (CSR)

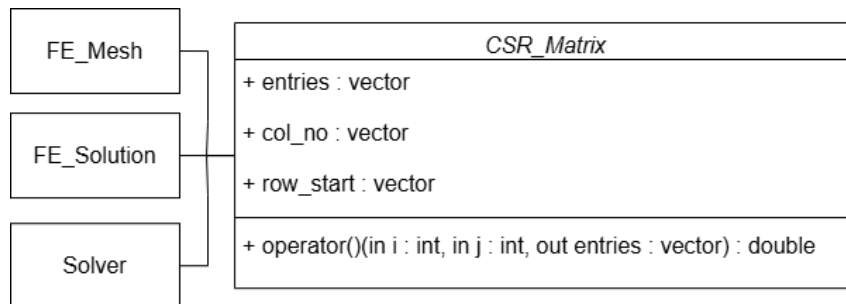


Figure 6: Compressed sparse row (CSR) matrix class diagram

The `CSR_Matrix` class is a *compressed sparse row* (CSR) matrix, which stores only the non-zero values. This is beneficial since most matrices involved in the finite element method

are sparse. It is made up of three vectors: `entries`, `col_no`, and `row_start`.

Example: *(conversion of full to compressed matrix)*

For this example we will use a common one dimensional stiffness matrix

$$A = \begin{bmatrix} 4 & -1 & 0 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & 0 & 0 \\ 0 & -1 & 4 & -1 & 0 & 0 \\ 0 & 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & 0 & -1 & 4 \end{bmatrix}$$

The process is fairly simple, we can fill the first two vectors by iterating through the matrix, appending the entry and column number for each non-zero value, then fill in the row start vector based on the entries of column number vector which begin each new row. Determining the sparsity pattern prior to knowing any values is more important to this implementation: this involves investigating the degrees of freedom for each element, and allocating space in the values and column numbers for each entry.

5.3.2 One and Two Dimensional Gaussian Quadrature

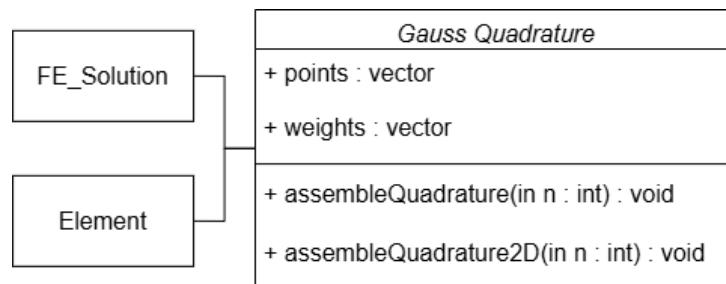


Figure 7: Gaussian quadrature class diagram

The GaussQuadrature class allows for an n_q -point Gaussian quadrature rule to be found. Lower order Gaussian quadrature are precomputed, and for $n_q > 5$ it generates points and weights along the line segment $[-1, 1]$ as the roots of the Legendre polynomials found using Newton's method. For the two dimensional reference element, a tensor product of the n_q -point quadrature is found and an affine transformation is used to map points to the required

triangle.

Gaussian quadrature is used to perform numerical integration as

$$\int_{\kappa^{\text{ref}}} f(x) \, dx \approx \sum_{k=1}^{n_q} w_k f(\xi_k),$$

where w_k denotes the weights, and ξ_k represents the quadrature points. An n_q -point quadrature can integrate polynomials up to degree $2n_q - 1$ exactly. This is used frequently throughout the implementation, specifically for stiffness and load integrals on each element. Additionally, the class extends to two dimensions as a tensor product of the one dimensional quadrature, then an affine mapping onto the reference triangle, with weights transformed by both the tensor product and the Jacobian of the mapping.

5.3.3 Helper Functions

The helper functions provide a simple suite of linear algebra tools for use in two dimensional calculations:

- `mapToPhysical`: Takes in coordinates of a physical triangle and a point ξ on the reference element, and returns the transformed point on the physical triangle.
- `computeAffineMatrix`: Takes in coordinates of a physical triangle and a 2×2 matrix, and fills in the matrix with the linear transformation part of the $x \mapsto Ax + b$ transformation
- `invertAffineMatrix`: Takes in an affine transformation matrix and returns the inverse

These functions are used throughout the local stiffness and load calculations, as well as during evaluation of the final solution.

5.4 Core Classes

Finally, we address the *core classes*, which are higher level classes facilitating the general control of the program. The finite element solution class controls the overall process of the code and contains the mesh, the solver, and the stiffness matrix and load vector. The finite

element mesh class holds the elements and nodes, and handles the assembly of the stiffness matrix and load vector. The Solver class is used to solve the linear system of equations to find the final output of the program.

5.4.1 Finite Element Mesh

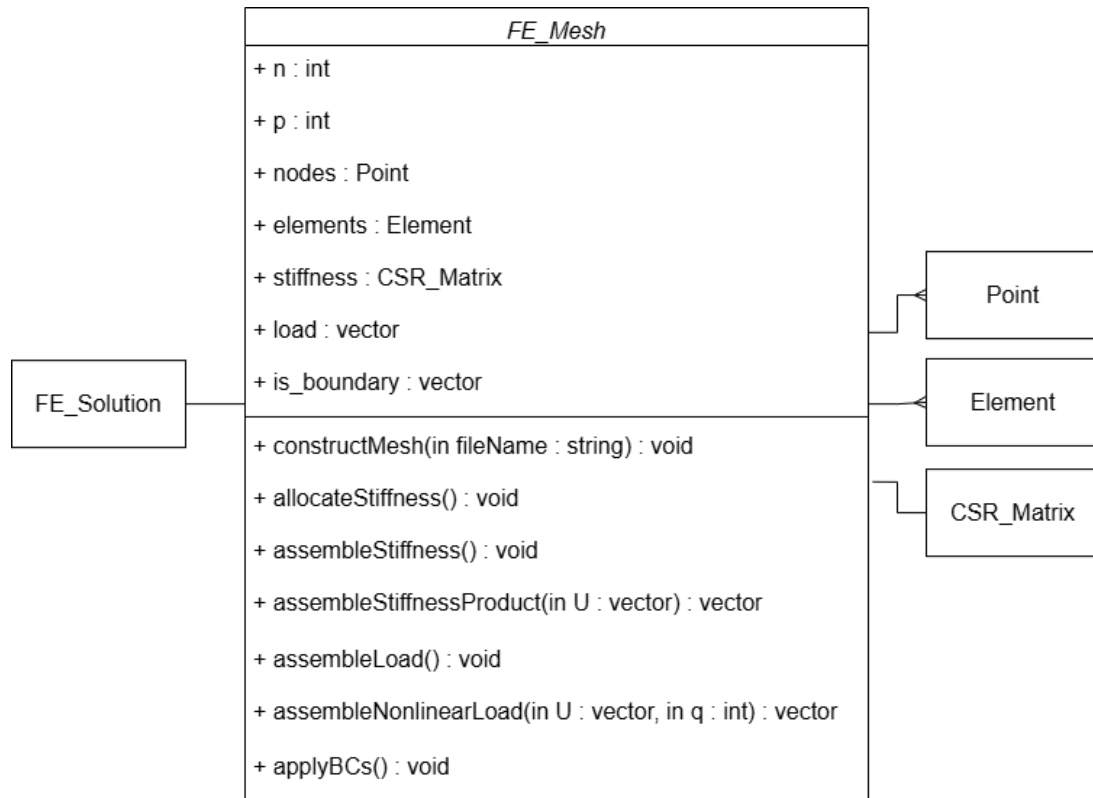


Figure 8: Finite element mesh class diagram

The **FE_Mesh** class represents the mesh, that is the discretisation of the continuous domain Ω into finite elements, which forms the basis for constructing the approximation space V_h . It encapsulates the number of elements (n), the polynomial degree (p), vectors of nodes and elements, and the global stiffness matrix and load vector. In one dimension Ω is discretised into n line segment elements over evenly spaced nodes, while in two dimensions the mesh is generated from `domain.node` and `domain.ele` files. The class also manages allocation of the compressed stiffness matrix and the assembly of local contributions into the global system, as well as application of boundary conditions. These functions are broken up into five methods seen in the class diagram in Figure 8.

Constructing the Mesh The method `constructMesh` is central to the implementation of the finite element method as it discretises the continuous domain Ω into the finite subdomains $\{\kappa\}_{i=1}^n$, such that

$$\Omega = \bigcup_{i=1}^n \kappa_i.$$

This partition is crucial for defining the space $V_h \subset V$. As the element size h tends to zero, V_h converges to V , yielding that the finite element solution approximates the true solution of the BVP.

In one dimension, Ω is the interval $[a, b]$ for which a uniform partition is found by defining nodes

$$x_i = a + i \cdot h, \quad i = 0, \dots, n,$$

where $h = \frac{b-a}{2}$ is the step size. Then each element is the line segment $[x_i, x_{i+1}]$. The degrees of freedom for an element are set as the global indices of its connectivity $[i, i + 1]$. In order to enrich the local approximation space, extra degrees of freedom are added corresponding to $p - 1$ bubble modes on each element.

In two dimensions, the mesh is generated by reading nodes and element connectivity data from files produced by Triangle (see Section 5.5). Each node is defined by its identifier and coordinates $\{id, x, y\}$, and each triangular element is defined by an ordered list of node identifiers. The three vertices provide the primary degrees of freedom. Then for higher order approximations, each edge is given $(p - 1)$ additional degrees of freedom. A mapping with an ordered pair of vertex IDs as a key ensures shared edges between elements are assigned consistent degree of freedom indices. Finally the interior of each triangle is enriched with $\frac{(p-1)(p-2)}{2}$ bubble modes by adding another degrees of freedom for each.

In both one and two dimensions, all elements are associated with a polynomial space of degree p , implementing the hierarchical basis defined in Section 5.2.2. The implementation for both cases can be seen in Algorithms 1 and 2.

Assembling Stiffness Matrix and Load Vector The mesh handles assembly of the stiffness matrix and load vector through the methods `assembleStiffness` and `assembleLoadVector`. These methods aggregate the local stiffness and load contributions such that for each element

Algorithm 1 An algorithm for creation of a 1D mesh

Require: $n \geq 1, p \geq 1$

```

1:  $h \leftarrow \frac{1}{n}$ 
2: for  $i$  in  $0, \dots, n+1$  do
3:    $nodes[i] \leftarrow i \cdot h$ 
4: end for
5:  $dof\_i \leftarrow n+1$  ▷ start of indexing for bubble dofs
6: for  $i$  in  $0, \dots, n$  do
7:    $ele\_nodes \leftarrow (nodes[i], nodes[i+1])$ 
8:    $dof \leftarrow (i, i+1)$ 
9:   for  $j$  in  $2, \dots, p+1$  do
10:     $dof[j] \leftarrow dof\_i$ 
11:    increment  $dof\_i$ 
12:   end for
13:    $elements[i] \leftarrow \{i, p, dof, ele\_nodes\}$ 
14: end for

```

κ , the mesh receives:

$$A_{ij}^\kappa = \int_\kappa \nabla \phi_i^{\text{phys}} \cdot \nabla \phi_j^{\text{phys}}, \quad b_i^\kappa = \int_\kappa f \phi_i^{\text{phys}},$$

where $\{\phi_i^{\text{phys}}\}$ denotes the local basis functions on κ . The number of degrees of freedom on each element is $p+1$ in one dimension, and $\frac{(p+1)(p+2)}{2}$ in two dimensions. These degrees of freedom are used to map the local contributions into the global system. This assembly process preserves the symmetry of the bilinear form and produces a symmetric global stiffness matrix.

Figure 9 shows a mesh with eight elements. Then for example element one has degrees of freedom $[3, 5, 6]$, so its local stiffness entries are added to those rows and columns in the the global stiffness matrix as:

$$\begin{bmatrix} & & & & & & \\ & * & & * & & * & \\ & * & & * & & * & \\ & * & & * & & * & \\ & & & & & & \end{bmatrix}. \quad (5.11)$$

Algorithm 2 An algorithm for creation of a 2D mesh

Require: $n \geq 1, p \geq 1$

```

1: for line in node_file do
2:   read in id, x, y, boundary_flag
3:    $nodes[i] \leftarrow \{id, x, y\}$ 
4:    $is\_boundary[i] \leftarrow boundary\_flag$ 
5: end for
6:  $dof\_i \leftarrow n + 1$ 
7: for line in elefile do
8:   read in id, i, j, k  $\triangleright i, j, k$  represent element connectivity
9:    $ele\_nodes \leftarrow (nodes[i], nodes[j], nodes[k])$ 
10:   $dof \leftarrow (i, j, k)$ 
11:  for  $j$  in  $0, \dots, 3$  do
12:     $edge \leftarrow ordered(dof[j], dof[(j + 1) \bmod 3])$ 
13:    if  $edge$  not seen before then
14:      for  $k$  in  $0, \dots, p - 1$  do
15:         $extra\_dof[k] \leftarrow dof\_i$ 
16:         $is\_boundary[dof\_i] \leftarrow$  both edges on boundary
17:         $dof\_i \leftarrow dof\_i + 1$ 
18:      end for
19:    else
20:      add edge dofs from shared boundary
21:    end if
22:     $dof \leftarrow extra\_dof$ 
23:  end for
24:  for  $j$  in  $0, \dots, \frac{(p-2)(p-1)}{2}$  do
25:     $dof \leftarrow dof\_i$ 
26:     $is\_boundary[dof\_i] \leftarrow false$ 
27:     $dof\_i \leftarrow dof\_i + 1$ 
28:  end for
29:   $elements[i] \leftarrow \{id, p, dof, ele\_nodes\}$ 
30: end for

```

Then the full matrix with all elements added in will be:

$$\begin{bmatrix}
 * & & & * & * & * \\
 & * & & * & * & * \\
 & & * & * & * & * \\
 * & * & * & * & * & * & * & * \\
 & * & * & * & * & & & \\
 * & & & * & * & * & & \\
 & & * & * & * & & * & \\
 * & * & & * & & & & *
 \end{bmatrix}. \tag{5.12}$$

Applying Boundary Conditions The `applyBCs` method applies Dirichlet boundary conditions by first finding all global degrees of freedom lying on the boundary of the domain. In

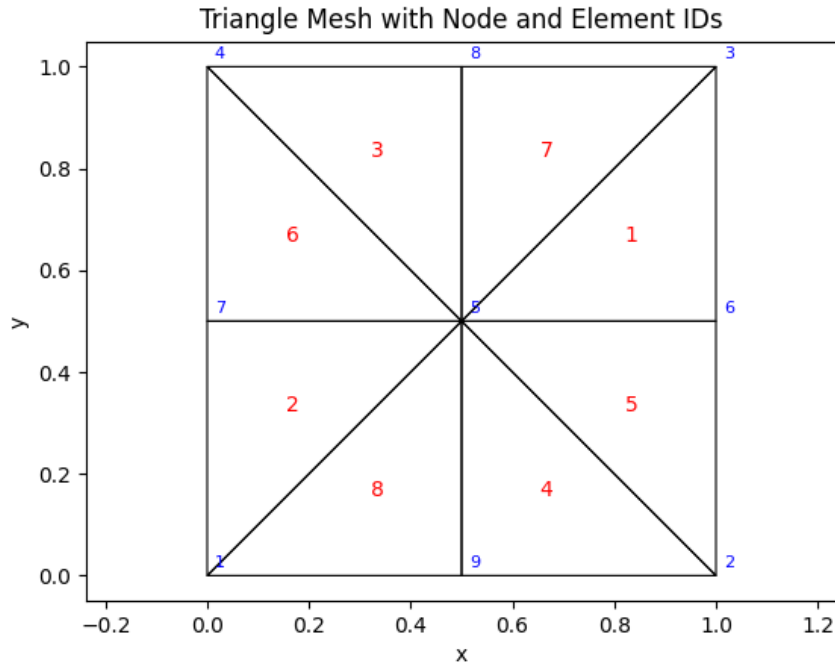


Figure 9: 2D mesh with eight elements

one dimension these are the end points of the interval, whereas in two dimensions they include a mixture of both vertices and edges along the domain boundary stored in a boolean vector `is_boundary` formed during mesh assembly. The enforcement is achieved by modifying the global stiffness matrix and load vector: rows and columns of boundaries are set to zero, and the diagonal to one. Then the corresponding entries in the load vector are set to the prescribed values.

Continuing the example from the previous section, entries $[1, 2, 3, 4, 6, 7, 8, 9]$ are boundaries, leaving the system:

$$\begin{bmatrix} * & & & & & & & \\ & * & & & & & & \\ & & * & & & & & \\ & & & * & & & & \\ & & & & * & & & \\ & & & & & * & & \\ & & & & & & * & \\ & & & & & & & * \end{bmatrix}. \quad (5.13)$$

For Semilinear Problems The class also has a two methods for aggregation of element contributions used for solving semilinear problems. `assembleStiffnessProduct` finds $a(U_n, v)$

as

$$\begin{aligned} a(U_n, v) &= \int_{\Omega} \nabla U_n \cdot \nabla v \, d\mathbf{x} \\ &= \sum_{\kappa} \int_{\kappa} \nabla U_n \cdot \nabla \phi^{\text{phys}} \, d\mathbf{x}, \end{aligned} \quad (5.14)$$

where $\int_{\kappa} \nabla U_n \cdot \nabla \phi^{\text{phys}} \, d\mathbf{x}$ is given by the element stiffness product routine. Boundary conditions are then applied based on the `is_boundary` vector, and the product is returned.

The `assembleNonlinearLoad` method similarly sums contributions from each element and applies boundary conditions for the non-linear portion of the problem.

$$\begin{aligned} b(U_n; v) &= \int_{\Omega} U_n^{2q+1} v \, d\mathbf{x} \\ &= \sum_{\kappa} \int_{\kappa} U_n^{2q+1} \phi^{\text{phys}} \, d\mathbf{x} \end{aligned} \quad (5.15)$$

These contributions are then used to form the right hand side during the Picard method iterative solution in the finite element solution class.

5.4.2 Finite Element Solution

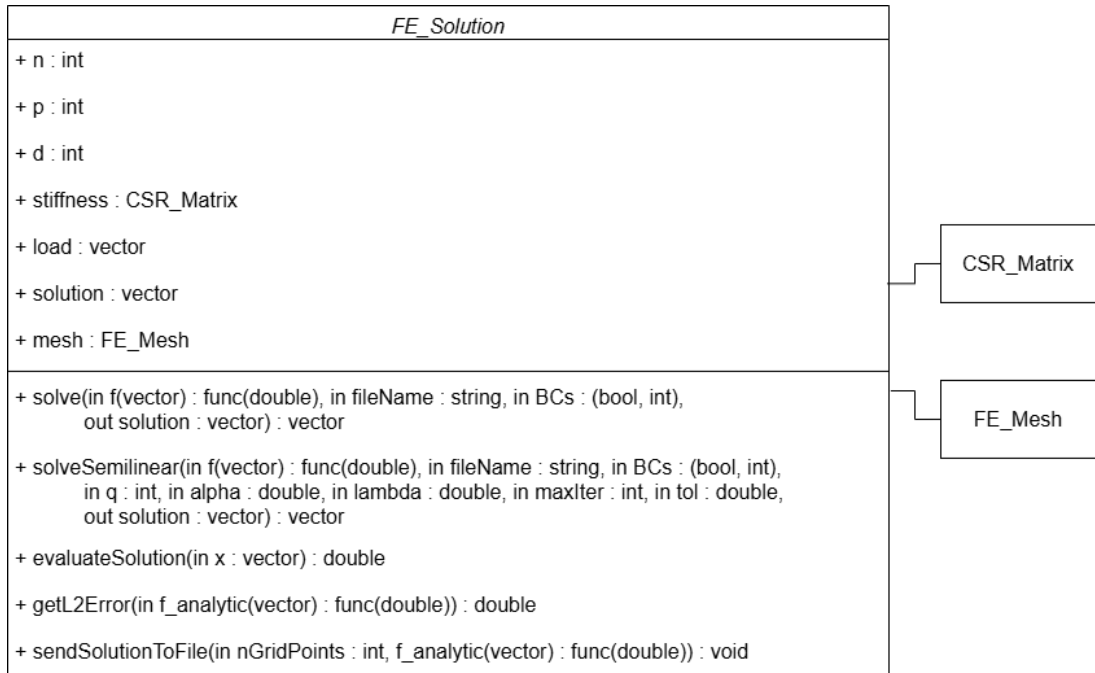


Figure 10: Finite element solution class diagram

The `FE_Solution` class is a high level controller class for the finite element program. It handles construction and solution of the discrete problem through a `FE_Mesh` object and a `Solver` object, as well as evaluation of the solution and error analysis.

Solving Linear Problems The `solve` method is used to orchestrate the solution process of the finite element method, in particular for variations of the Poisson problem:

$$-\Delta u = f, \quad \partial\Omega = 0,$$

where f is a function passed in to `solve`. Then the process is implemented as follows:

1. **Mesh construction:** The method begins by invoking the `constructMesh` method of its member `mesh`, which discretises the continuous domain Ω into finite elements, and establishes the degrees of freedom of each element. Then space is allocated for the stiffness matrix in compressed sparse row format.
2. **System assembly:** Stiffness and load are found on each element, and assembled into the global system $Au = b$ based on their degrees of freedom. Then boundary conditions are applied
3. **System solution:** A `Solver` object is created with the stiffness matrix A and load vector b and the system is solved for u_h , using Eigen. This is justified by Theorem 1 guaranteeing a unique solution for positive definite A . Finally, the solution is returned.

Solving Semilinear Problems The `solveSemilinear` method follows a similar process for semilinear problems of the form:

$$-\Delta u + \lambda u^{2q+1} = f, \quad \partial\Omega = 0,$$

with fixed positive integer q , positive constant λ , and function f passed as parameters to the method. Then the process up to step two is the same as the linear case, then the process follows:

1. **Set initial guess:** Setup initial guess U_n , and preallocate U_{n+1}
2. **Construct the right hand side vector:** The stiffness product $a(U_n, v)$ and nonlinear form $b(U_n; v)$ are collected from the mesh and construct

$$RHS = (1 - \alpha)a(U_n, v) + \alpha [b - \lambda b(U_n; v)],$$

with b being the global load found earlier, and α being the relaxation parameter.

3. **Solve new system and check for convergence:** The saved global stiffness matrix and the calculated right hand side are sent to the solver object, and solved by the direct method for U_{n+1} . Then the L^2 -norm $\|U_{n+1} - U_n\|_{L^2}$ is found and compared to a tolerance parameter

Evaluation and Analysis Once a solution is found, there are a few methods used for analysis.

- Firstly the `evaluateSolution` method computes the value of the finite element solution $u_h(x)$ for an arbitrary point $x \in \Omega$. It finds the point in an element, then reconstructs the continuous approximation from the discrete representation through:

$$u_h(x) = \sum_{i=1}^{N_{dof}} u_i \phi_i(x),$$

with solution vector $\mathbf{u}$ and polynomial basis $\{\phi_i\}$ on V_h .

- Then the `getL2Error` method computes the L^2 -norm error between the approximation u_h and the analytical solution u defined by f_{analytic} . The L^2 error

$$\|u - u_h\|_{L^2(\Omega)} = \left(\int_{\Omega} (u(x) - u_h(x))^2 \right)^{1/2},$$

is approximated via Gaussian quadrature. This error metric provides a quantitative measure of the approximation's accuracy and is used to verify the theoretical convergence rates.

- Finally, the `sendSolutionToFile` method exports the solution and error data to an external CSV. This is done by evaluating the approximate solution u_h at an input number of uniformly spaced points on Ω (along the line segment in one dimension, or a uniform grid in two dimensions). This is used for post-processing such as visualisation of the solution and comparison to the analytical solution.

5.4.3 Solver

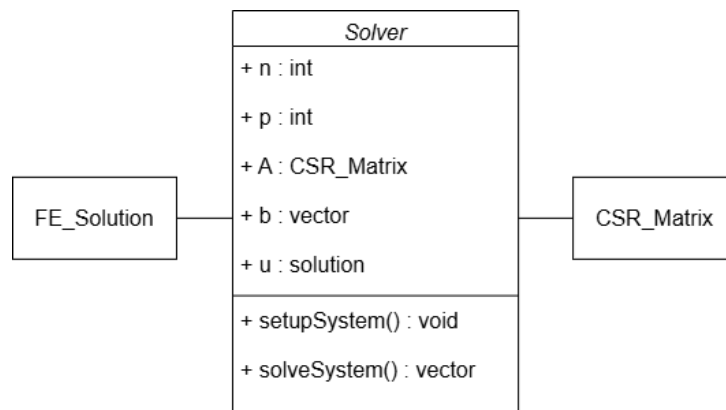


Figure 11: Solver class diagram

The `Solver` class is responsible for solving the linear system

$$Au = b$$

for u . Since this is a symmetric, positive definite matrix, a Cholesky factorisation is used from Eigen's library of efficient sparse matrix routines. The class is instantiated with a CSR stiffness matrix A and a load vector b . These are first converted to `Eigen::SparseMatrix` and `Eigen::VectorXd` respectively, and then the system is solved by employing `Eigen::SimplicialLLT`. This performs Cholesky decomposition as

$$A = LL^T,$$

where L is a lower triangular matrix. It then solves the system through

$$Ly = b \quad L^T u = y.$$

This has the benefit of requiring around half the computation and memory compared to a full LU factorisation.

5.5 Supplementary Items

Finally, there are a few supplementary items to lend credit to. Eigen is a linear algebra library used for the direct sparse LLT Cholesky solver in this implementation. And Triangle is a mesh generation program used for generating a triangulated mesh based on a set of nodes and connectivity.

6 Convergence Studies

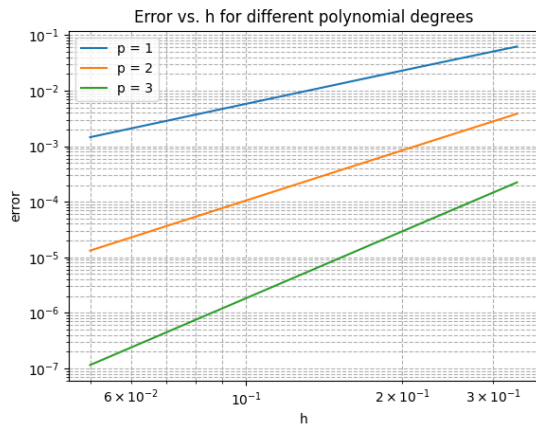
In order to assess the accuracy of a finite element method, we consider the convergence of the approximation u_h to the exact solution u . In particular, we can observe the convergence with respect to increasing the mesh resolution (h -refinement), and increasing the polynomial degree (p -refinement). In this section we will consider classical error estimates, and compare them to the results from the implementation.

6.1 h -refinement

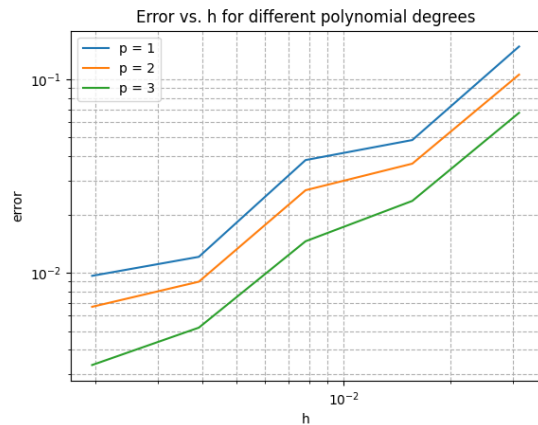
Under h -refinement, the polynomial degree is fixed at p and the maximum element size h is decreased. For a sufficiently smooth solution, a common error bound given in [16] in the L^2 norm is

$$\|u - u_h\|_{L^2(\Omega)} \leq Ch^{p+1},$$

that is the convergence rate increases with a finer mesh. Numerical experiments in this implementation show the error diminishing at a rate consistent with the theoretical bounds. This is illustrated in one dimension in Figure 12a, and in two dimensions in Figure 12b.



(a) Convergence of 1D solution to exact solution under h -refinement



(b) Convergence of 2D solution to exact solution under h -refinement

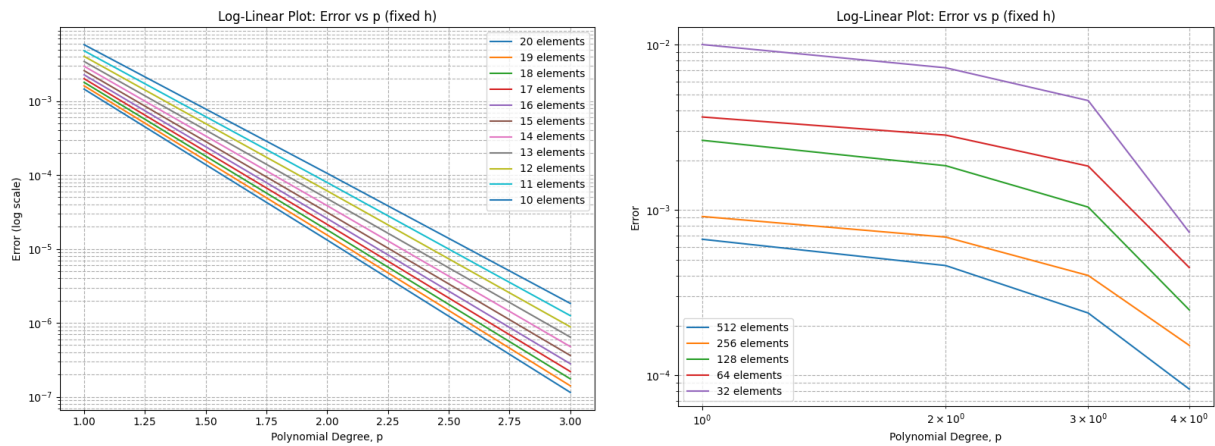
Figure 12: h -refinement convergence plots

We see that in the one dimensional case, the error estimate is satisfied with gradients matching the theory exactly for each p . In the two dimensional case, however, while the error is overall lower for each p , the expected straight line is not exhibited, and the rate of

convergence is only marginally faster for higher p , not the polynomial convergence required by theory.

6.2 p -refinement

Under p -refinement, the mesh resolution is fixed at h and the polynomial degree p is increased. For a sufficiently smooth solution, exponential convergence is observed in the L^2 norm in the order $\mathcal{O}(e^{-p})$.



(a) Convergence of 1D solution to exact solution under h -refinement (b) Convergence of 2D solution to exact solution under h -refinement

Figure 13: p -refinement convergence plots

We see this bound clearly satisfied by the one dimensional case, with faster convergence for finer meshes. While in the two dimensional case, the solution does converge, however not in the required order.

7 Applications

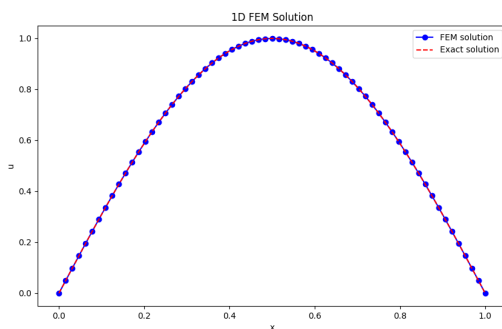
PDEs are ubiquitous throughout science and engineering, appearing in fluid flow, structural deformation, and heat conduction to name a few. Since their analytical solutions are often infeasible to calculate, the finite element method is invaluable to provide numerical solutions to these problems. In this section we will see the FEM implementation applied to a range of benchmark problems.

7.1 Poisson Equation in One Dimension

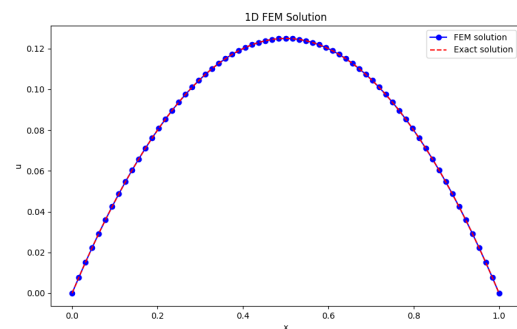
We begin with the one dimensional Poisson equation,

$$-u'' = f(x), \quad u(0) = u(1) = 0.$$

This is a classic problem, which could be used to model the bending of a beam under load or heat distribution along a rod. The forcing function $f(x)$ can be changed according to the problem. Shown in Figure 14a is the forcing function $f(x) = \pi^2 \sin(\pi x)$, which has known solution $u(x) = \sin(\pi x)$, and the implementation solution is shown to match that exactly with four elements and a polynomial degree of three. Then in Figure 14b we see a constant forcing function $f(x) = 1$, with analytic solution $u(x) = \frac{x(1-x)}{2}$, where approximation is perfect again.



(a) 1D FEM approximation for trigonometric forcing function



(b) 1D FEM approximation for constant forcing function

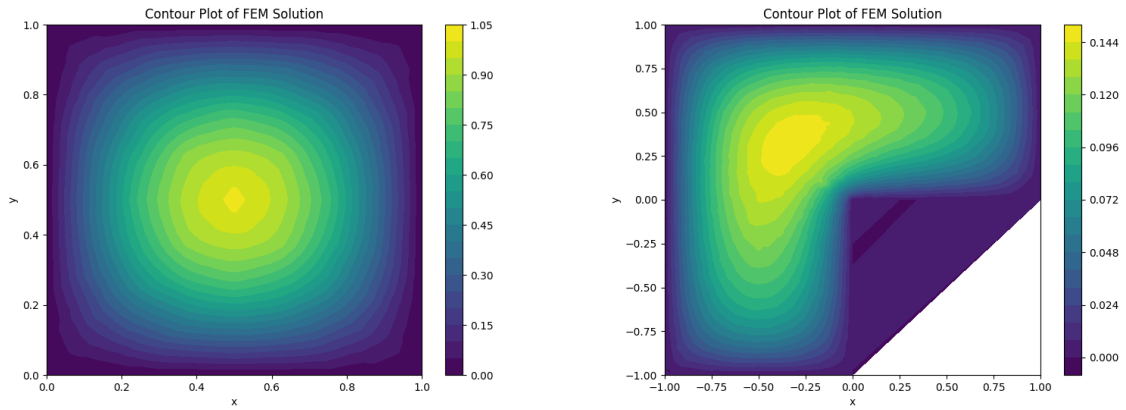
Figure 14: Applications of FEM in 1D

7.2 Poisson Equation in Two Dimensions

We now consider the Poisson problem in two dimensions

$$-\Delta u = f(x, y).$$

First we see a trigonometric forcing function $f(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y)$, which has an oscillatory solution $u(x, y) = \sin(\pi x) \sin(\pi y)$. In figure 15a, we see the smooth contour plot for the approximate solution with 128 elements, and polynomial degree three. Then with a constant forcing function, in Figure 15b we see the contour plot of the solution on an L-shaped domain. This result is particularly interesting as it is a solution which is difficult to attain analytically.



(a) 2D FEM approximation for trigonometric forcing function

(b) 2D FEM approximation for constant forcing function

Figure 15: Applications of FEM in 2D

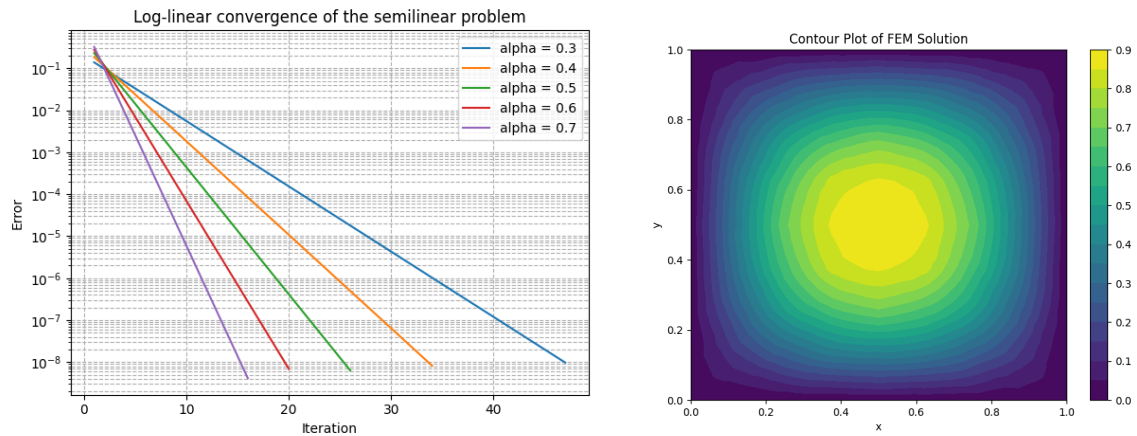
7.3 Semilinear Problem

Finally we consider a semilinear problem of the form

$$-\Delta u + \lambda u^{2q+1} = f(x, y),$$

where the nonlinear u^{2q+1} term introduces some complexity into the model. In the iterative solver implemented for this type of problem, the relaxation parameter α can be varied: the

convergence can be seen in Figure 16a for an array of values. The solution using forcing function $f(x, y) = 2\pi^2 \sin(\pi x) \sin(\pi y)$ is shown in Figure 16b. For most values of λ and q , this exhibits similar behaviour to the fully linear problem, however the peak begins to flatten out for very high values as seen in Figure 16b.



(a) Convergence rates of Picard solver for varying α (b) Semilinear FEM approximation for trigonometric forcing function

Figure 16: Semilinear problems using FEM

8 Conclusions

This dissertation has developed and implemented a high-order object-oriented finite element framework aimed at solving second order linear and semilinear elliptic BVPs in one and two dimensions. A general theoretical foundation was developed, drawing from a number of classical and contemporary research, and it was demonstrated how concepts such as hierarchical modal bases can be implemented in C++. Numerical experiments demonstrated converging solutions in both one and two dimensions, with the one dimensional case achieving the prescribed $\mathcal{O}(h^{p+1})$ error bound. In two dimensions, while the convergence was evident from the applications, did not achieve the exact error bounds required. The inclusion of a Picard solver accounted for a degree of nonlinearity in the range of problems available; also achieving the required exponential convergence to a solution for a range of relaxation parameters. Overall, this project has effectively bridged the gap between the mathematical foundations and the computational considerations which make up finite element theory.

Further Work Moving forward, there are a few natural avenues for future work on this project. The first is implementing Neumann and mixed boundary conditions described in Section 3.2.1, which would have the benefit of allowing much more complex problems to be modelled, as they specify the derivative at the boundary, rather than fixed values. Implementation of this would involve an adjustment to the global load vector to incorporate the integrated boundary terms, as well as handling this effect across element boundaries.

A second interesting extension would be into *hp*-adaptivity. This is an adaptive mesh refinement and polynomial degree refinement based on *a posteriori* error estimates, which optimises accuracy while minimising the computational cost. To implement this, a robust method for error estimation would be required, as well as a data structure to store the varying polynomial orders across elements, both of which are detailed in [7].

Finally, implementation of fully nonlinear problems (with the highest order terms appearing nonlinearly) would expand the range of problems to include common equations, such as Navier-Stokes for fluid flow. Implementation of this would require a different iterative solver, and careful treatment of second derivatives to ensure convergence.

A Code

All code for the implementation can be found at

`https://github.com/dwulfse/FEMwithCPP`.

References

- [1] Philippe G. Ciarlet. *The finite element method for elliptic problems*. Number 40 in Classics in applied mathematics. Society for Industrial and Applied Mathematics (SIAM, 3600 Market Street, Floor 6, Philadelphia, PA 19104), Philadelphia, Pa, 2002.
- [2] Susanne C. Brenner and L. Ridgway Scott. *The Mathematical Theory of Finite Element Methods*, volume 15 of *Texts in Applied Mathematics*. Springer, New York, NY, 2008.
- [3] O Axelsson, VA Barker, and DJ Benson. Finite Element Solution of Boundary Value Problems: Theory and Computation. Classics in Applied Math, Vol. 35. *Applied Mechanics Reviews*, 55(3), June 2002.
- [4] Barna Szabó and Ivo Babuska. *Finite Element Analysis: Method, Verification and Validation*. Wiley, 06 2021.
- [5] Claudio Canuto, M. Youssuff Hussaini, Alfio Quarteroni, and Thomas A. Zang. *Spectral Methods: Fundamentals in Single Domains*. Scientific Computation. Springer, Berlin, Heidelberg, 2006.
- [6] George Karniadakis and Spencer Sherwin. *Spectral/hp Element Methods for Computational Fluid Dynamics*. Oxford University Press, June 2005.
- [7] Pavel Šolín, Karel Segeth, and Ivo Doležel. *Higher-order finite element methods*. Studies in advanced mathematics. Chapman & Hall/CRC, Boca Raton, FL, 2004.
- [8] Mats G. Larson and Fredrik Bengzon. *The Finite Element Method: Theory, Implementation, and Applications*, volume 10 of *Texts in Computational Science and Engineering*. Springer, Berlin, Heidelberg, 2013.
- [9] Haim Brezis. *Functional Analysis, Sobolev Spaces and Partial Differential Equations*. Springer, New York, NY, 2011.
- [10] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations*. Other Titles in Applied Mathematics. Society for Industrial and Applied Mathematics, January 2007.

- [11] Claes Johnson. *Numerical solution of partial differential equations by the finite element method*. Cambridge Univ. Press, Cambridge, 5. printing edition, 1994.
- [12] T. Sorgente, S. Biasotti, G. Manzini, and M. Spagnuolo. A Survey of Indicators for Mesh Quality Assessment. *Computer Graphics Forum*, 42(2):461–483, 2023.
- [13] Endre Süli and D. F. Mayers. *An introduction to numerical analysis*. Cambridge University Press, Cambridge ; New York, 2003.
- [14] Lloyd N. Trefethen and David Bau. *Numerical linear algebra*. Society for Industrial and Applied Mathematics, Philadelphia, 1997.
- [15] Yanchen He, Paul Houston, Christoph Schwab, and Thomas P. Wihler. Exponential Convergence of hp -ILGFEM for semilinear elliptic boundary value problems with monomial reaction, April 2024.
- [16] Ajay Harish. What is Convergence in Finite Element Analysis? SimScale, January 2017.